



Universidad del Valle de Guatemala

Facultad de Ingeniería
Departamento de Matemáticas

Teoría de la Computación Sección 20

Proyecto #2

Proyecto No. 2 – Algoritmo CYK (Cocke–Younger–Kasami)

Grupo # 4

Integrante 1: Pablo Daniel Barillas Moreno, Carné No. 22193

Integrante 2: Hugo Daniel Barillas Ajin, Carné No. 23556

Profesor: Ing. Bidkar Alexander Pojoy Corzo

Índice

1. Resumen	2
2. Objetivos y alcance	2
2.1. Objetivos del proyecto	2
2.2. Alcances	2
3. Diseño de la aplicación	3
3.1. Arquitectura y flujo	3
3.2. Modelo de datos y decisiones	3
3.3. Interfaz de usuario	3
4. Teoría y algoritmos	4
4.1. Definiciones	4
4.2. Pipeline de conversión a CNF	4
4.3. Algoritmo CYK	5
5. Gramática del proyecto (CFG original)	5
6. Discusión: obstáculos, decisiones y recomendaciones	5
6.1. Obstáculos y soluciones	5
6.2. Riesgos, supuestos y trazabilidad	6
6.3. Recomendaciones y mejoras	6
7. Ejemplos y pruebas realizadas	6
7.1. Criterios de validación	6
7.2. Casos de prueba (muestras)	6
7.3. Tabla resumen de resultados	7
7.4. Pruebas automatizadas	7
7.5. Árbol de parseo	7
8. Correctitud y complejidad	8
8.1. Correctitud del pipeline CNF	8
8.2. Correctitud de CYK (boceto)	8
8.3. Complejidad	8
9. Guía de uso y reproducibilidad	8
9.1. Requisitos	8
9.2. Ejecución	8
9.3. Descargas	8
10. Referencias	10

1. Resumen

Este proyecto implementa un *pipeline* completo para decidir si una oración en inglés pertenece al lenguaje generado por una gramática libre de contexto (CFG). La solución:

- Convierte la CFG dada a **Forma Normal de Chomsky (CNF)** mediante: (i) eliminación de ε -producciones, (ii) eliminación de unitarias, (iii) eliminación de símbolos inútiles (no generadores e inalcanzables), (iv) aumento del símbolo inicial cuando $S \Rightarrow^* \varepsilon$, y (v) binarización/separación de terminales.
- Ejecuta el **algoritmo CYK** (*Cocke–Younger–Kasami*), técnica de programación dinámica con complejidad $O(n^3|G|)$, que decide pertenencia y permite reconstruir al menos un **árbol de parseo**.
- Entrega **SÍ/NO**, **tiempo de ejecución** y el **árbol sintáctico** como texto y en formato `.dot` (Graphviz), cumpliendo los requisitos del enunciado.

Tecnologías. Python 3, Streamlit (UI), `pandas` (tablas/estadísticas), Graphviz/`dot` (árboles).

2. Objetivos y alcance

2.1. Objetivos del proyecto

- **Simplificación de gramáticas y conversión a CNF.**
- **Implementación de CYK** con tabla triangular y *backpointers*.
- **Salidas requeridas:** SÍ/NO, tiempo de validación y **parse tree**.
- **Documentación:** diseño, discusión (obstáculos/recomendaciones), ejemplos y pruebas.

2.2. Alcances

Se soporta la gramática del enunciado (inglés simple con *NP, VP, PP*, etc.). La UI permite cargar una CFG desde texto o usar la gramática del proyecto. El árbol reconstruye una derivación válida (primera encontrada); la enumeración de múltiples árboles queda como trabajo futuro.

3. Diseño de la aplicación

3.1. Arquitectura y flujo

La solución se compone de cuatro módulos principales:

1. **Grammar**: representación de CFG (V, Σ, P, S) , alta de producciones, validación y exportación.
2. **CNFConverter**: pipeline de conversión a CNF (cinco pasos) con trazas y verificación final.
3. **CYKParser**: algoritmo CYK con tabla triangular, índices inversos y *backpointers*.
4. **UI (Streamlit)**: selección/visualización de gramática, conversión, ejecución de CYK, descargas y pruebas.

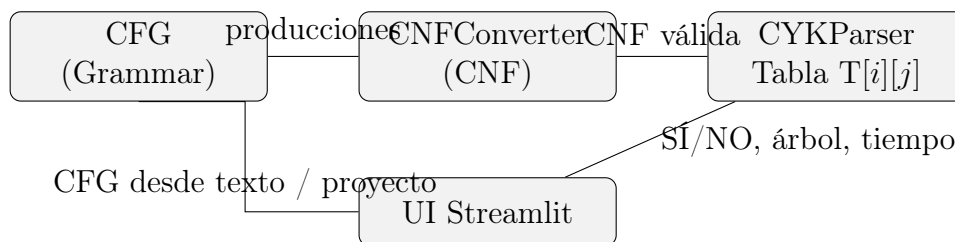


Figura 1: Flujo alto nivel: $\text{CFG} \rightarrow \text{CNF} \rightarrow \text{CYK}$.

3.2. Modelo de datos y decisiones

- Producciones: `Dict[str, List[List[str]]]`; cada RHS es lista de símbolos.
- Σ se *recalcula* como todo símbolo en RHS que no sea no terminal ni ε .
- Variables auxiliares en CNF: $X_0, X_1, \dots$ garantizan no colisión en V .
- Tabla CYK: `table[i][j]` es un conjunto (**set**) de no terminales (operaciones $O(1)$).
- Índices inversos para eficiencia:

$$(B, C) \mapsto \{A \mid A \rightarrow BC \in P\}, \quad a \mapsto \{A \mid A \rightarrow a \in P\}.$$

- **Backpointers**: para $A \in \text{table}[i][j]$, se almacena (**production**, B, C, k) o (**terminal**, a) y se reconstruye un árbol por DFS.

3.3. Interfaz de usuario

La UI ofrece:

- Selección de gramática (*Proyecto*, *Aritmética*, *Desde texto*).
- Conversión guiada a CNF con *snapshots* intermedios de P .
- Análisis CYK con: SÍ/NO, tiempo (s), tabla triangular y árbol (texto + descarga `.dot`).
- Batería de pruebas y métricas (pasados/ % de éxito).

4. Teoría y algoritmos

4.1. Definiciones

Una CFG es $G = (V, \Sigma, P, S)$. En CNF, todas las producciones son de la forma $A \rightarrow a$ o $A \rightarrow BC$ (con $A, B, C \in V, a \in \Sigma$), y opcionalmente $S \rightarrow \varepsilon$.

4.2. Pipeline de conversión a CNF

(1) Eliminación de ε -producciones. Se computa el conjunto de *anulables* por punto fijo:

```
nullable := {}  
repetir  
    cambio := falso  
    para (A -> ) en P:  
        si = o ( no vacía y X: Xnullable):  
            agregar A a nullable; cambio := verdadero  
hasta que no haya cambio
```

Luego, para cada $A \rightarrow \alpha$, se generan todas las RHS α' eliminando subconjuntos de símbolos anulables ($\alpha' \neq \varepsilon$ salvo el caso especial del símbolo inicial).

(2) Eliminación de unitarias $A \rightarrow B$. Clausura transitiva de unitarias:

```
U := {(A,B) | A->B P unitarias}  
repetir  
    para (A,B) en U, (B,C) en U: agregar (A,C) a U  
hasta cerradura  
para cada (A,B) en U:  
    copiar a A todas las unitarias de B si no estaban  
eliminar unitarias de P
```

(3) Símbolos inútiles. Dos fases: (i) no generadores (desde terminales hacia arriba) y (ii) no alcanzables (desde S hacia abajo). Se filtran de P .

(4) Aumento del símbolo inicial. Si $S \in \text{nullable}$, introducir $S_0 \rightarrow S \mid \varepsilon$ y redefinir $S := S_0$. Re-ejecutar (2) para retirar la unitaria $S_0 \rightarrow S$.

(5) Binarización y separación de terminales. Para $A \rightarrow X_1 X_2 \dots X_k$ con $k > 2$, crear variables auxiliares que *encadenen* el RHS. Si hay terminales en reglas binarias, introducir variables T_a tales que $T_a \rightarrow a$.

4.3. Algoritmo CYK

Entrada: cadena $w = w_1 \dots w_n$, gramática en CNF.

```
para j=1..n:
  T[j][j] := { A | A->w_j P }
para longitud = 2..n:
  para i = 1..n-longitud+1:
    j := i+longitud-1
    T[i][j] :=
      para k = i..j-1:
        para B T[i][k], C T[k+1][j]:
          T[i][j] := T[i][j] { A | A->BC P }
Resultado: aceptar si S T[1][n]
```

Correctitud (boceto). **Solidez:** Si $A \in T[i][j]$, entonces $A \Rightarrow^* w_i \dots w_j$ (inducción en la longitud). **Compleitud:** Si $A \Rightarrow^* w_i \dots w_j$ en la CNF, la derivación se descompone en producciones binarias y terminales que CYK cubrirá por construcción.

Complejidad. Tiempo $O(n^3|G|)$ y espacio $O(n^2)$.

5. Gramática del proyecto (CFG original)

```
S → NP VP
VP → VP PP | V NP | cooks | drinks | eats | cuts
PP → P NP
NP → Det N | he | she
V → cooks | drinks | eats | cuts
P → in | with
N → cat | dog | beer | cake | juice | meat | soup | fork
  | knife | oven | spoon
Det → a | the
```

6. Discusión: obstáculos, decisiones y recomendaciones

6.1. Obstáculos y soluciones

- **Detección de anulables sin falsos positivos.** Se usó punto fijo y la regla “todos los símbolos en RHS ya anulables”.
- **Reintroducción de unitarias tras aumentar S .** Tras $S_0 \rightarrow S \mid \varepsilon$ se re-ejecuta la eliminación de unitarias.
- **Terminales en producciones binarias.** Se introducen variables X_a por terminal para cumplir CNF.
- **Reconstrucción de árbol.** Se guarda un *backpointer* por no terminal y subcadena; se reconstruye la *primera* derivación válida (suficiente para la rúbrica).
- **Tokens fuera de vocabulario y mayúsculas.** Recomendamos normalizar a minúsculas y remover puntuación simple previo al análisis.

6.2. Riesgos, supuestos y trazabilidad

- *Ambigüedad*: si la CFG es ambigua, CYK detecta aceptación pero no enumera todos los árboles; se documenta como limitación consciente.
- *Tamaños*: el costo cúbico vuelve lentas oraciones muy largas; se acota el uso a $n \lesssim 50$ para UI fluida.
- *Auditoría*: la app muestra *snapshots* de P por fase del pipeline y valida CNF al final.

6.3. Recomendaciones y mejoras

- Añadir una fase de *normalización* (minúsculas y *stripping* de signos) para aceptar ejemplos con mayúscula inicial y punto final.
- Enumerar múltiples árboles en gramáticas ambiguas (no requerido).
- Exportar la documentación desde la UI en `.md` junto con el `.dot`.

7. Ejemplos y pruebas realizadas

7.1. Criterios de validación

- Verificación automática de CNF tras el pipeline.
- Casos **válidos** e **inválidos** (con/ sin PP y con/ sin pronombres).
- Reporte de **SÍ/NO**, **tiempo** (solo fase CYK) y **árbol** cuando aplica.

7.2. Casos de prueba (muestras)

Válidos.

- she eats a cake with a fork
- the cat drinks the beer
- he cuts the meat with a knife
- a dog eats the soup
- she drinks the juice in the oven
- the cat cooks

Inválidos.

- she eat cake (*verbo y falta de determinantes*)
- dog eats soup (*faltan determinantes*)
- cat the eats (*orden incorrecto*)
- beautiful cat eats (*“beautiful” no está en la gramática*)
- she (*oración incompleta*)

7.3. Tabla resumen de resultados

Entrada	Esperado	Obtenido	Tiempo [s]	Notas
she eats a cake with a fork	SÍ	SÍ	0,01	Árbol generado
the cat drinks the beer	SÍ	SÍ	0,01	Árbol generado
he cuts the meat with a knife	SÍ	SÍ	0,01	Árbol generado
a dog eats the soup	SÍ	SÍ	0,01	Árbol generado
the cat cooks	SÍ	SÍ	0,01	Árbol generado
she eat cake	NO	NO	0,00	Tokenización co- rrecta
dog eats soup	NO	NO	0,00	Falta Det
cat the eats	NO	NO	0,00	Orden inválido
beautiful cat eats	NO	NO	0,00	<i>beautiful</i> no está en Σ
she	NO	NO	0,00	Incompleta

Cuadro 2: Resultados observados (tiempos típicos en equipo estándar).

7.4. Pruebas automatizadas

La app incluye una batería con estado PASS/FAIL y porcentaje de éxito, además de pruebas especiales:

- **Gramática que acepta ε :** se comprueba que la cadena vacía se acepta cuando $S \Rightarrow^* \varepsilon$ tras el aumento del símbolo inicial.
- **Gramática aritmética** (opcional): para validar el *pipeline* y CYK en un dominio diferente.

7.5. Árbol de parseo

Para entradas aceptadas, se genera:

- Árbol textual (reglas aplicadas por niveles).
- Archivo `parse_tree.dot` para Graphviz (`dot -Tpng parse_tree.dot -o tree.png`).

8. Correctitud y complejidad

8.1. Correctitud del pipeline CNF

Cada fase es finita y monótona (el conjunto de producciones se transforma reduciendo formas inválidas o introduciendo únicamente variables auxiliares); el resultado satisface CNF por inspección de formas $A \rightarrow a$ y $A \rightarrow BC$ (y opcionalmente $S \rightarrow \varepsilon$).

8.2. Correctitud de CYK (boceto)

Por inducción en la longitud de subcadenas:

- Base: $j = i$. $T[i][i]$ contiene A si $A \rightarrow w_i$ es una producción.
- Paso: si $A \Rightarrow BC \Rightarrow^* w_i \dots w_k w_{k+1} \dots w_j$, entonces $B \in T[i][k]$ y $C \in T[k+1][j]$, por hipótesis inductiva; por tanto $A \in T[i][j]$.

8.3. Complejidad

La triple anidación por (i, j, k) da $O(n^3)$; el factor $|G|$ aparece en el número de reglas binarias consideradas mediante el índice $(B, C) \mapsto A$. El espacio es $O(n^2)$ por la tabla triangular.

9. Guía de uso y reproducibilidad

9.1. Requisitos

Python 3.10+, `streamlit`, `pandas`. Opcional: `Graphviz` para renderizar `.dot` en PNG.

9.2. Ejecución

1. Instalar dependencias: `pip install streamlit pandas`.
2. Ejecutar: `streamlit run CYK_Algorithm_Implementation.py`
3. Seleccionar “Gramática del Proyecto (PDF)” y probar ejemplos de la Sección 7.

9.3. Descargas

- `grammar_cnf.txt` con las producciones en CNF.
- `parse_tree.txt` y `parse_tree.dot` del árbol.

Anexo A: Extracto CNF (ejemplo)

La aplicación permite descargar el conjunto completo; un extracto típico luce así (variables auxiliares X_i generadas automáticamente):

```
S -> NP VP
VP -> V NP
VP -> VP PP
PP -> P NP
NP -> Det N
NP -> he
NP -> she
V -> cooks | drinks | eats | cuts
P -> in | with
N -> cat | dog | beer | cake | juice | meat | soup | fork | knife | oven | spoon
% Tras binarizar/normalizar (ejemplo ilustrativo)
X0 -> drinks
VP -> X0
...
```

Anexo B: Mapeo explícito a la rúbrica

Criterio de la rúbrica	Evidencia en este documento/implementación
Implementación del algoritmo de simplificación (CNF) (7 pts)	Secciones: Teoría y Pipeline CNF (Sección 5), Correctitud CNF (Sección 8.1), Extracto CNF (Anexo A). Código: clase CNFConverter con fases (1)–(5).
Implementación del algoritmo CYK (7 pts)	Secciones: Algoritmo CYK (Sección 5.2), Correctitud/Complejidad (Sección 8), Ejemplos/Pruebas (Sección 7). Código: clase CYKParser , tabla triangular, backpointers, medición de tiempo.
Documentación formal (1 pt)	Este informe: diseño (Sección 4), discusión (Sección 6), ejemplos/pruebas (Sección 7), guía de uso (Sección 9).

10. Referencias

- Cocke–Younger–Kasami algorithm. (s.f.). *Wikipedia*. Recuperado el 13 de octubre de 2025, de https://en.wikipedia.org/wiki/CYK_algorithm
- Context-free grammar. (s.f.). *Wikipedia*. Recuperado el 13 de octubre de 2025, de https://en.wikipedia.org/wiki/Context-free_grammar
- Forme normale de Chomsky. (s.f.). *Wikipédia*. Recuperado el 13 de octubre de 2025, de https://fr.wikipedia.org/wiki/Forme_normale_de_Chomsky
- Algoritmo CYK. (s.f.). *Wikipedia, a enciclopedia libre*. Recuperado el 13 de octubre de 2025, de https://gl.wikipedia.org/wiki/Algoritmo_CYK
- DOT language—Reference. (s.f.). *Graphviz*. Recuperado el 13 de octubre de 2025, de <https://graphviz.org/doc/info/lang.html>
- User Guide: graphviz package. (s.f.). *Read the Docs/Graphviz*. Recuperado el 13 de octubre de 2025, de <https://graphviz.readthedocs.io/en/stable/manual.html>
- Streamlit Documentation. (s.f.). *Streamlit*. Recuperado el 13 de octubre de 2025, de <https://docs.streamlit.io/>
- CYK Algorithm in Theory of Computation. (s.f.). *TutorialsPoint*. Recuperado el 13 de octubre de 2025, de <https://www.tutorialspoint.com/cyk-algorithm-in-theory-of-computation>